Backpropagation in MLP, Transfer Learning in CNN and RL Report

Machine Learning for Robotics

January 15, 2025

Contents

1 Executive Summary **2**

2 Regression **3**
2.1 Task A: Deriving the Backpropagation Algorithm for a 1-3-1 Network 3
2.1.1 Forward Propagation 3
2.1.2 Backpropagation 4
2.2 Training and Testing the 1-3-1 Neural Network using SGD 5
2.3 Training and Testing the 1-3-1 Neural Network using the ADAM Algorithm . . 6
2.4 Task D: Exploring different Network Configurations using TensorFlow 7
2.4.1 Network Architectures and Training 7
2.4.2 Results 8
2.4.3 Final Performance Metrics 8
2.4.4 Discussion 8
2.4.5 Conclusion 9
2.5 Task E: Using just ReLU in the hidden layer to approximate the data 9

3 Classification **11**
3.1 Task A: Deriving the Backpropagation Algorithm 11
3.1.1 Forward Pass 11
3.1.2 Backward Pass 11
3.2 Task B: Training, and testing with stochastic gradient descent 12
3.2.1 Generalisation to Matrix Notation and SGD Implementation 12
3.2.2 SGD Results and Discussion 13
3.3 Task C: Training and Testing with other Optimisers 14
3.3.1 Implementing SGD with Momentum 14
3.3.2 Implementing the Adam Optimiser 14
3.3.3 Results and Comparison to Vanilla SGD 15
3.3.4 Conclusion on Optimiser Experiments 16
3.4 Task D: Exploring different Network Configurations using TensorFlow 16
3.4.1 Network Architectures and Training Setup 16
3.4.2 Experiment Results for Network Architecture Comparison 17
3.4.3 Analysis for Network Architecture Comparison 17
3.4.4 Conclusion for Network Architecture Comparison 18

4 Transfer Learning **19**
4.1 Task A: Dataset Preparation 19
4.2 Task B and C: Transfer Learning Implementation and Results 19
4.2.1 Network Architecture and Modifications 19
4.2.2 Training Configuration and Performance 19
4.2.3 Classification Analysis 20
4.2.4 Discussion 21
4.3 Task D: Hyperparameter Optimisation Analysis 21
4.3.1 Optimiser Performance Analysis 21
4.3.2 Cross-Architecture Comparison 22

5 Q-Learning **23**

6 Conclusion **31**

1 Executive Summary

This report explores four important areas in the field of Machine Learning: regression, classification, transfer learning, and Q-learning. Each section looks into both theoretical foundations and practical implementations, demonstrating an understanding of these concepts.

Regression and Classification utilise Neural Networks (NNs), comprising layers of interconnected nodes to process data, identifiy patterns, and make predictions. These networks handle non-linear relationships and can output probabilities for multiple classes, allowing accurate classification and regression tasks.

Transfer Learning leverages pre-trained Convolutional Neural Networks (CNNs) to classify new image datasets. CNNs are suited for high-dimensional data like images due to their ability to reduce the number of learnable weights, enhancing computational efficiency and performance in classification tasks.

Q-Learning, a reinforcement learning technique, enables an agent to make optimal decisions through trial and error interactions with its environment. This section implements tabular Q-learning in a grid-world scenario, showing how agents learn to maximise rewards by selecting the best actions on learned Q-values

Task 1 involves implementing and training a 1-3-1 Multilayer Perceptron (MLP) from scratch using backpropogation and stochastic gradient descent (SGD). The performance of this custom implementation is compared with MATLAB's Neural Network Toolbox and further evaluated using the ADAM optimisation algorithm. Additionally, the impact of varying network architecture parameters, such as the numbers of layers and nodes, is experimented with using the toolbox

Task 2 focuses on classifying the Iris dataset using a 4-5-3-3 MLP implemented from scratch. The network employs SGD for training and its performance is contrasted with alternative architectures and training algorithms using TensorFlow and Keras

Task 3 applies transfer learning techniques using pre-trained CNNs, specifically AlexNet and GoogLeNet, to classify 75 fruit images across five categories. This tasks shows the significance of learning rate and optimiser adjustments in CNN performance, examining how varying these parameters affect classification outcomes.

Task 4 implements tabular Q-learning within a grid-world environment to determine the optimal action for each cell, maximising cumulative rewards. The report details the Q-value updates over the first three iterations of the initial episodes, providing insights into the learning process of the agent.

2 Regression

Machine Learning (ML) is widely used to analyse relationships between a dependent variable and independent variables. This involves finding a function that best describes these relationships. Regression analysis, a key ML algorithm, is applied across sectors like Real Estate (predicting house prices) and Finance (forecasting prices from historical data). The Neural Network used here is a 1-3-1 network: 1 input node, 3 hidden nodes with tanh activation, and 1 output node with linear activation. Accurate forward propagation and backpropagation using stochastic gradient descent (SGD) are essential for its regression analysis.

2.1 Task A: Deriving the Backpropagation Algorithm for a 1-3-1 Network

Backpropagation begins with forward propagation using random weights and biases, producing an output with error. It minimises errors by updating weights and biases after each epoch to approach the optimal line of best fit. The 1-3-1 Neural Network is depicted below. The total error, calculated as the sum of squared errors (SSE), will use sequential mode, finding instantaneous squared errors, explained later.

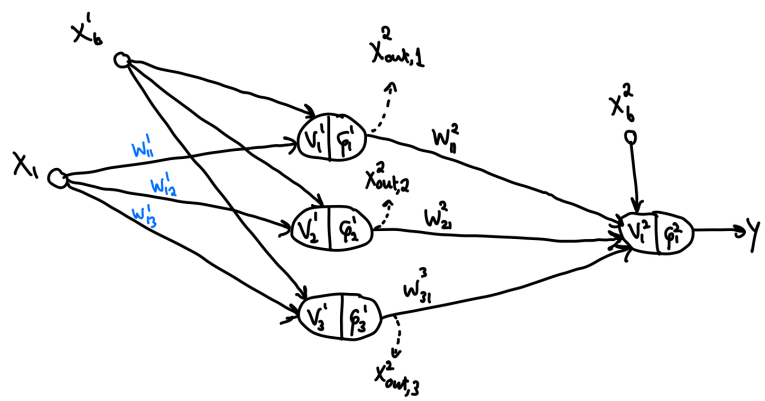


Figure 1: Neural Network Diagram

2.1.1 Forward Propagation

In Fig. 1, the input contains an index on the subscript and the parameters contain indices in the superscript and the subscript. The superscript defines the node number, starting from one at the top node. The subscript defines the layer number in the neural network. The input and biases are inserted into the three nodes of the hidden layer. The input is multiplied by three different weights w_{11}^1, w_{12}^1 and w_{13}^1 before being inputted into the three nodes in the hidden layer. The values v_1^1, v_2^1 and v_3^1 are:

$$v_1^1 = w_{11}^1 x_1 + x_b^1, v_2^1 = w_{12}^1 x_1 + x_b^1, v_3^1 = w_{13}^1 x_1 + x_b^1 \tag{1}$$

The values v_1^1, v_2^1 and v_3^1 are then inserted into the activation functions ϕ_1^1, ϕ_2^1 and ϕ_3^1. From the various types of activation functions, $\phi(v) = tanh(v)$ will be used for each node in the hidden layer. This activation function is centered at zero which gives average values of the hidden layer output, $x_{out,1}^2, x_{out,2}^2$ and $x_{out,3}^2$ closer to zero resulting in faster convergence during training. The equations below show the values of these outputs:

$$x_{out,1}^2 = \phi_1^1(v_1^1), x_{out,2}^2 = \phi_2^1(v_2^1), x_{out,3}^2 = \phi_3^1(v_3^1) \tag{2}$$

These outputs are then multiplied by the corresponding three weights in layer two, w_{11}^2, w_{21}^2 and w_{31}^2. They are then summed up to together, including the bias, x_{b1}^2, and inputted to the activation function ϕ_1^2:

$$v_1^2 = w_{11}^2\, x_{\text{out},1}^2 + w_{21}^2\, x_{\text{out},2}^2 + w_{31}^2\, x_{\text{out},3}^2 + x_b^2, \qquad y_1 = \phi_1^2(v_1^2) \tag{3}$$

The activation function at the output node will be a linear function, therefore, $\phi_1^2(v_1^2) = v_1^2$. Replacing $v_1^2, x_{out,1}^2, x_{out,2}^2$ and, $x_{out,3}^2$ with the variables in the first layer:

$$y_1 = w_{11}^2\, \phi_1^1\big(w_{11}^1\, x_1 + x_b^1\big) + w_{21}^2\, \phi_2^1\big(w_{12}^1\, x_1 + x_b^1\big) + w_{31}^2\, \phi_3^1\big(w_{13}^1\, x_1 + x_b^1\big) + x_b^2. \tag{4}$$

With this function we will now assume appropriate random values of the weights, input and biases to calculate a random error which will be minimised by our backpropagation algorithm, which finds the optimal SSE.

$$SSE = \frac{1}{2}(d_1 - y_1)^2 \equiv SSE = \frac{1}{2}(d_1 - \phi_1^2(v_1^2))^2 \tag{5}$$

2.1.2 Backpropagation

SGD will be used for backpropagation in sequential mode, updating weights after each data point. This is faster than batch mode and better at avoiding local minima in noisy cost functions. The cost function, an instantaneous squared error, is shown in Eq. 5. This method adjusts weights in the direction of steepest descent, as shown below for w_{11}^2 between the first hidden layer node and the output node:

$$w_{11,new}^2 = w_{11,old}^2 - \eta \frac{\partial SSE}{\partial w_{11}^2} \tag{6}$$

General equation, where index i depicts the node number and j the layer number:

$$w_{ij,new}^2 = w_{ij,old}^2 - \eta \frac{\partial SSE}{\partial w_{ij}^2} \tag{7}$$

At the optimal SSE with respect to w_{11}^2, the partial derivative becomes zero, meaning the weight stops changing, indicating the optimal weight is found. Using the chain rule, we calculate the partial derivative, since other functions depend on w_{11}^2. From Eq. 3, w_{11}^2 is a variable in v_1^2, allowing us to rewrite the partial derivative as:

$$\frac{\partial SSE}{\partial w_{11}^2} = \frac{\partial SSE}{\partial v_1^2}\frac{\partial v_1^2}{\partial w_{11}^2} \tag{8}$$

We then find the partial derivative value by finding the two partial derivatives. From Eq. 3:

$$\frac{\partial v_1^2}{\partial w_{11}^2} = x_{out,1}^2, \qquad \frac{\partial SSE}{\partial v_1^2} = -(d_1 - \phi_1^2(v_1^2))\frac{\partial \phi_1^2}{\partial v_1^2} \tag{9}$$

Where $\frac{\partial \phi_1^2}{\partial v_1^2} = 1$. Inputting these derivative functions, we get the SGD equation for w_{11}^2:

$$w_{11,new}^2 = w_{11,old}^2 + \eta(d_1 - \phi_1^2(v_1^2))x_{out,1}^2 \tag{10}$$

The same method applies to the other three weights between the hidden layer nodes, the output node, and the bias. To complete backpropagation, we derive SGD equations for the

three weights between the input and hidden layers and the biases. Focusing on w^1_{11}, the equation is shown below:

$$w^1_{11,new} = w^1_{11,old} - \eta \frac{\partial SSE}{\partial w^1_{11}} \tag{11}$$

Differentiation chain rule finds the gradient of SSE with respect to w^1_{11}. Here, w^1_{11} is in v^1_1 (Eq. 1), v^1_1 is in $x^2_{out,1}$ (Eq. 2), $x^2_{out,1}$ is in v^2_1 (Eq. 3), and v^2_1 is in SSE (Eq. 5). The partial derivative can be rewritten as:

$$\frac{\partial SSE}{\partial w^1_{11}} = \frac{\partial SSE}{\partial v^2_1} \frac{\partial v^2_1}{\partial x^2_{out,1}} \frac{\partial x^2_{out,1}}{\partial v^1_1} \frac{\partial v^1_1}{\partial w^1_{11}} \tag{12}$$

Lastly, we find the values of each of these partial derivatives in order to substitute them into Eq. 11: $\frac{\partial SSE}{\partial v^2_1} = -(d_1 - \phi^2_1(v^2_1))$, as calculated previously, $\frac{\partial v^2_1}{\partial x^2_{out,1}} = w^2_{11}$. The third partial derivative requires the derivative of $tanh$, which is $\frac{\partial \phi}{\partial v} = 1 - (\phi(v)^2$. Therefore: $\frac{\partial x^2_{out,1}}{\partial v^1_1} = \frac{\partial \phi^1_1}{\partial v^1_1} = 1 - (\phi^1_1(v^1_1))^2$, $\frac{\partial v^1_1}{\partial w^1_{11}} = x_1$, considering Eq. 1. Substituting these values into Eq. 11:

$$w^1_{11,new} = w^1_{11,old} + \eta(d_1 - \phi^2_1(v^2_1))w^2_{11}(1 - (\phi^1_1(v^1_1))^2)x_1 \tag{13}$$

The same method applies for the other weights and biases to find their optimal values for regression analysis with minimal SSE. The final equations will resemble Eq. 10 and Eq. 13. The Neural Network with updated weights and biases will then be tested on unseen data via forward propagation. With the backpropagation equations established, we can now apply these principles in Task B to train and test the 1-3-1 network on real data.

2.2 Training and Testing the 1-3-1 Neural Network using SGD

The aim of this task is to create a MATLAB code which will train and test the network derived in Task a for the regression of the data shown below.

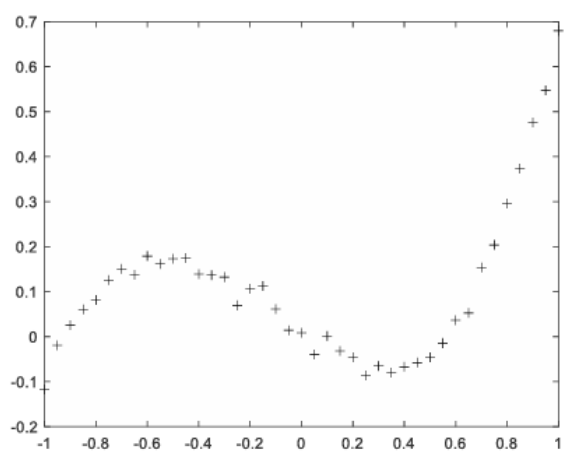
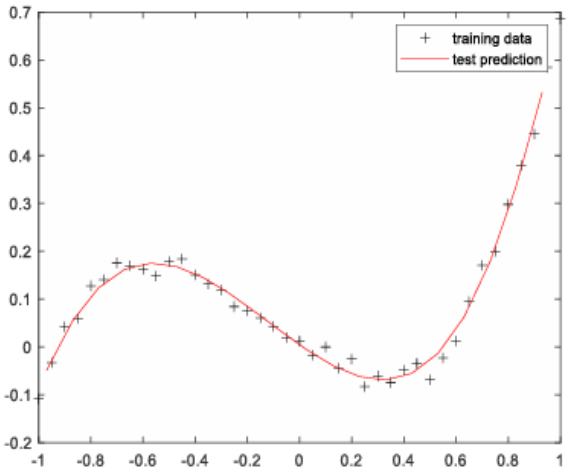
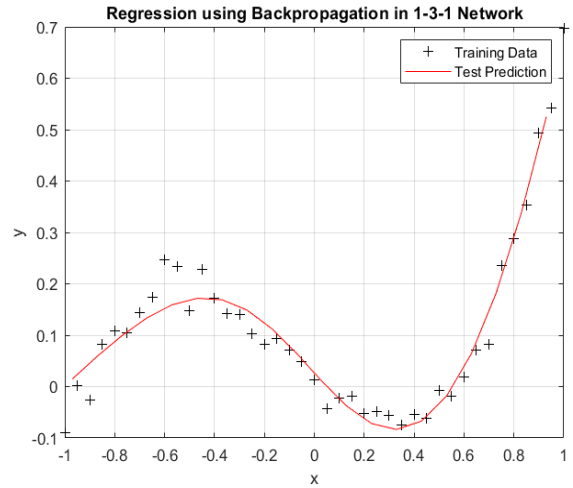


Figure 2: Training data for regression

Using the method in Section 2.1.2, we update all weights and biases for regression via backpropagation and test them on unseen data using forward propagation, as in Section 2.1.1. The expected outcome, shown in Fig.3a, matches the MATLAB output in Fig.3b.



(a) Expected output for regression



(b) Regression output

Figure 3: Expected output and Actual Output for regression

Comparing Fig. 3a and Fig. 3b, the network is well-trained, with nearly identical regression lines. Proper tuning of the learning rate and epochs can improve accuracy. A high learning rate η risks non-convergence, while too few epochs may prevent sufficient weight updates for optimal SSE. Results suggest the learning rate and epochs must exceed specific thresholds, determined through trial and error, to achieve satisfactory outcomes. While SGD yielded satisfactory results, another training algorithm called ADAM will be explored to identify potential improvements.

2.3 Training and Testing the 1-3-1 Neural Network using the ADAM Algorithm

The aim of this task is to use a different training algorithm to train and test our network for regression analysis of the same dataset and compare the results.
The ADAM training algorithm combines the Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp).

$$\boxed{m(t+1) = \beta_m m(t) + (1-\beta_m)\left(\frac{\partial(L)}{\partial(w)}\right)}, \boxed{\hat{m}(t+1) = \frac{m(t+1)}{1-\beta_m^{t+1}}}$$

$$\boxed{v(t+1) = \beta_v v(t) + (1-\beta_v)\left(\frac{\partial(L)}{\partial(w)}\right)^2}, \boxed{\hat{v}(t+1) = \frac{v(t+1)}{1-\beta_v^{t+1}}}$$

$$\boxed{w(t+1) = w(n) - \frac{\eta}{\sqrt{\hat{v}(t+1)+\varepsilon}} \cdot \hat{m}(t+1)}$$

Figure 4: ADAM Algorithm Equations

The SGD algorithm can be slow, and when a dataset has many local minima, it may get stuck in a suboptimal minimum instead of the global one. The ADAM algorithm addresses this by adjusting the learning rate based on previous gradients. In Fig.4, the first equation represents the first moment estimate. If $\frac{\partial SSE}{\partial w}$ has the same sign in consecutive iterations, the first moment term increases, adjusting the weight's sign. The second moment term (velocity) controls the learning rate size. Both terms are bias-corrected ($\hat{m}$ and $\hat{v}$) to prevent starting at zero, improving training in early epochs.

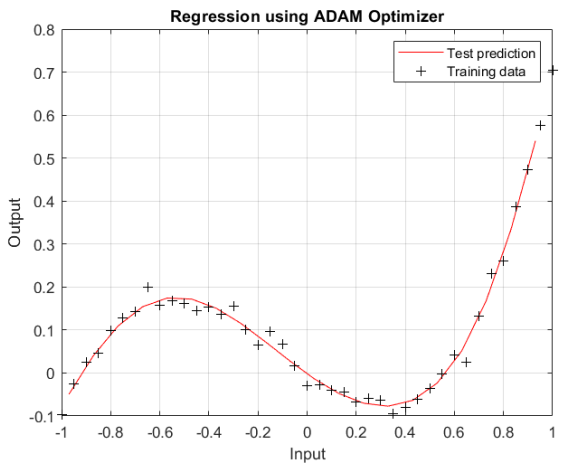


Figure 5: ADAM Algorithm Graph

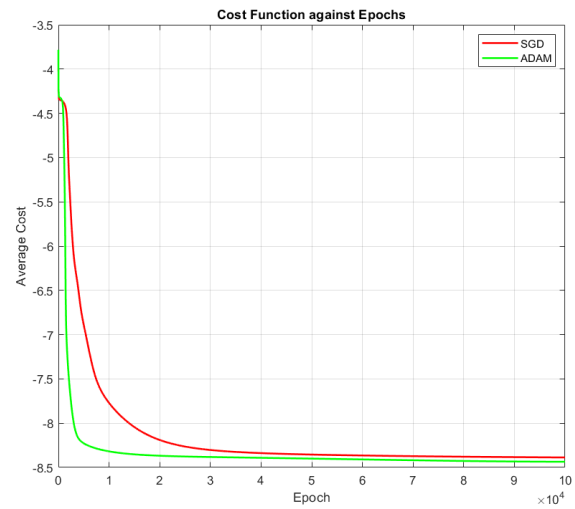


Figure 6: Cost functions of SGD and ADAM algorithms plotted together

Fig.5 illustrates the result of the ADAM algorithm, while Fig.6 compares the cost functions of ADAM and SGD. Fig.6 highlights that ADAM's cost function decreases significantly faster than SGD's, stabilising after 5,000 epochs compared to SGD's 22,000. This indicates that ADAM converges more efficiently. Both algorithms succeed in regression analysis; however, ADAM offers faster convergence due to its combination of momentum and adaptive learning rates (Fig.4). Momentum smooths weight updates by incorporating past gradients, while adaptive learning rates adjust step sizes to ensure steady progress. Overall, ADAM enhances training stability and efficiency, achieving a better-fitting regression line, particularly for non-linear relationships. Having explored the effect of training algorithms on regression performance, the role of the architecture of a neural network will be evaluated next.

2.4 Task D: Exploring different Network Configurations using TensorFlow

This task aims to identify the most effective neural network architecture for regression using Keras and TensorFlow for rapid implementation. The focus is on evaluating the impact of two key architectural features on performance: **1) The Number of Layers and Nodes** and **2) The Activation Functions in the Output and Hidden Layers**

2.4.1 Network Architectures and Training

For this task, five network architectures were tested to evaluate their optimal performance: the original 1-3-1 network from Task b), a smaller 1-1-1 network with a linear output and tanh activation, a 1-3-1 network with a linear output and ReLU activation, a deep 1-4-8-16-1 network using ReLU, sigmoid, and tanh activations in the hidden layers with a tanh output, and a large 1-64-1 network with ReLU activation and a linear output.

To assess these different neural network structures in terms of optimal performance, a hyperparameter search was employed to find the optimal learning rate for every model. This learning parameter is then used to gain the best regression result possible from every model and analyse the convergence rate of these models, gaining insight on the performance of the different networks in terms of both speed and performance. The following are the resulting optimal learning parameters for each model, visualised in form of a table:

Architecture	1-3-1 (original)	1-1-1 (smaller)	1-3-1 (ReLU)	1-64-1 (large)	1-4-8-16-1 (deep)
Best LR	0.081920	0.04096	0.081920	0.163840	0.04096

Table 1: Optimal learning rates for regression determined through grid search for each neural network architecture.

2.4.2 Results

The training process yielded the following accuracy and loss curves:

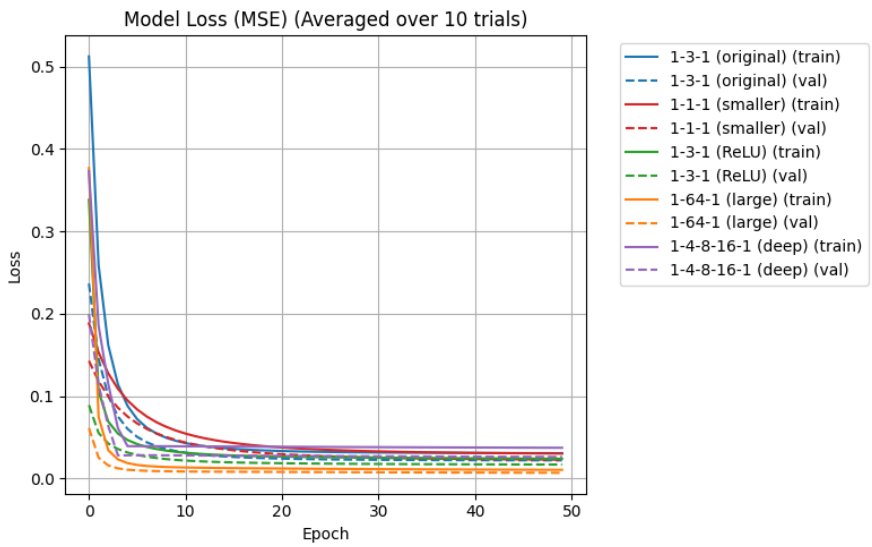


Figure 7: Mean Squared Error (MSE) across epochs for the proposed neural network (NN) models. Note: "val" represents the test dataset in this context.

2.4.3 Final Performance Metrics

For evaluating regression performance, the mean absolute error (MAE) is included in Table 2 due to its robustness to outliers and interpretability in real-world applications. Unlike the mean squared error (MSE), which heavily penalises larger errors, MAE provides a more balanced view of prediction accuracy by averaging absolute deviations.

Architecture	Training MAE	Test MAE	Training Loss	Test Loss
1-3-1 (original)	0.1460 ± 0.0006	0.1313 ± 0.0005	0.0307 ± 0.0001	0.0219 ± 0.0001
1-1-1 (smaller)	0.1450 ± 0.0074	0.1326 ± 0.0074	0.0302 ± 0.0010	0.0227 ± 0.0012
1-3-1 (ReLU)	0.1178 ± 0.0170	0.1033 ± 0.0147	0.0240 ± 0.0088	0.0168 ± 0.0062
1-64-1 (large)	**0.0894 ± 0.0026**	**0.0718 ± 0.0028**	**0.0104 ± 0.0007**	**0.0068 ± 0.0005**
1-4-8-16-1 (deep)	0.1451 ± 0.0048	0.1314 ± 0.0086	0.0372 ± 0.0024	0.0261 ± 0.0024

Table 2: Performance comparison of various neural network architectures for regression, presented as mean ± standard deviation (std) over 10 trials. The best values in each column are highlighted in bold. Note: "Loss" refers to the Mean Squared Error (MSE) in this context.

2.4.4 Discussion

Having found the optimal parameter for every model, the remaining difference in performance between the models can be tracked back to the difference in structure. This is the aspect where the focus will lie in the following.

Impact of Network Size: The large model consistently outperformed others across all metrics, achieving a training MAE of 0.0894 ± 0.0026 and a testing MAE of 0.0718 ± 0.0028, with strong generalisation and reliability. In contrast, the smaller 1-1-1 model showed comparable performance to the original, with a slightly worse testing MAE (0.1326 ± 0.0074 vs. 0.1313 ± 0.0005) but a slightly better average training MAE. However, the smaller model had a higher standard deviation (0.0074 vs. 0.0006), indicating poorer worst-case performance.

The large model also demonstrated greater robustness, with low standard deviations for both training (0.0026) and testing (0.0028) MAE, second only to the original model. In contrast, the 1-3-1 ReLU model showed far higher variability (0.0170 for training and 0.0147 for testing). Additionally, the large model converged faster, reaching convergence around epoch 10, compared to epoch 20 for the original and 1-3-1 ReLU models and epoch 30 for the small model. This suggests that larger models train more efficiently.

Activation Function Effects: Replacing the hidden layer's tanh activation in the original 1-3-1 model with ReLU improved performance, reducing the training MAE to 0.1178 ± 0.0170 and testing MAE to 0.1033 ± 0.0147, representing a performance boost of 19.3% and 21.3%, respectively. However, this improvement came at the cost of reliability, as the ReLU model's standard deviations (0.0170 for training and 0.0147 for testing) were about 30 times higher than the original. Both models reached convergence at around epoch 20.

Impact of Network Depth: The deep 1-4-8-16-1 model showed similar regression performance to the original model, with marginally better training and slightly worse testing MAE. However, it was significantly less reliable, with standard deviations 8 times higher for training (0.0048) and 17 times higher for testing (0.0086). Despite its lower robustness, the deep model had the fastest convergence, fully converging in about 4 epochs, as can be seen in Figure 7, significantly outpacing all other models.

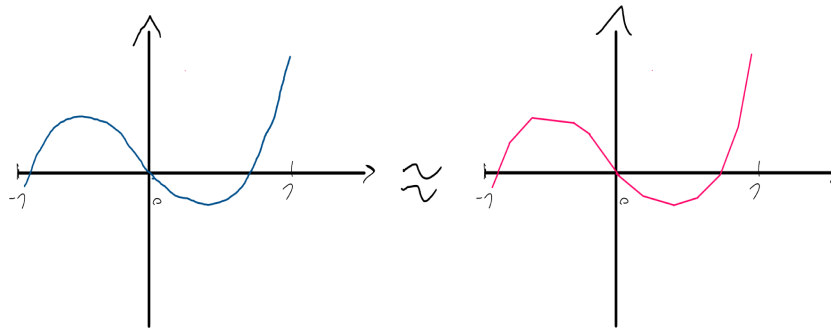
2.4.5 Conclusion

Using a comprehensive hyperparameter search to optimise learning rates, this study systematically evaluated five neural network architectures for regression, ranging from simple to large and deep configurations. The selection of an optimal neural network structure ultimately depends on the specific priorities of the task at hand. For maximum regression accuracy, the large 1-64-1 model stands out as the best option, offering superior performance metrics and robustness. If rapid convergence is a higher priority and a modest trade-off in accuracy is acceptable, the deep 1-4-8-16-1 model may be more suitable due to its ability to converge significantly faster. Alternatively, for scenarios where stability is critical and slight performance compromises are acceptable, the original 1-3-1 model provides a reliable choice. Among these options, the large 1-64-1 model offers the best overall balance, combining the highest accuracy, twice the convergence speed of the original 1-3-1 model, and a low standard deviation, making it the clear winner in this comparison. This model will also be important in the next section to prove an important point. This model will also play a key role in the next section to demonstrate a crucial point.

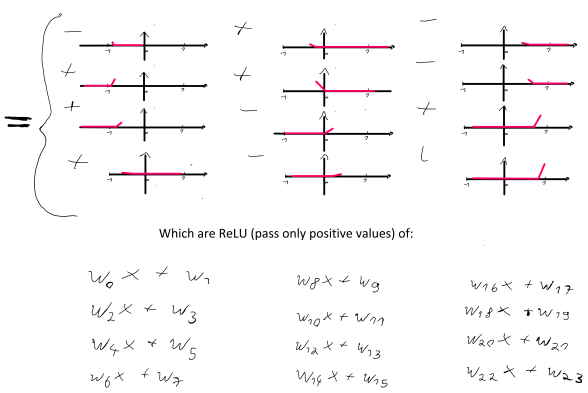
2.5 Task E: Using just ReLU in the hidden layer to approximate the data

ReLU (Rectified Linear Unit) has become widely used as an activation function, particularly valued for regression tasks due to its ability to capture complex relationships with efficient training dynamics. Defined as $ReLu(x) = max(0, x)$, it introduces non-linearity in the network, which is essential for modeling complex, non-linear data patterns. In this section, a sketch will demonstrate how using only ReLU in the hidden layer effectively approximates the target function, illustrating its strengths in accurately modeling non-linear data.

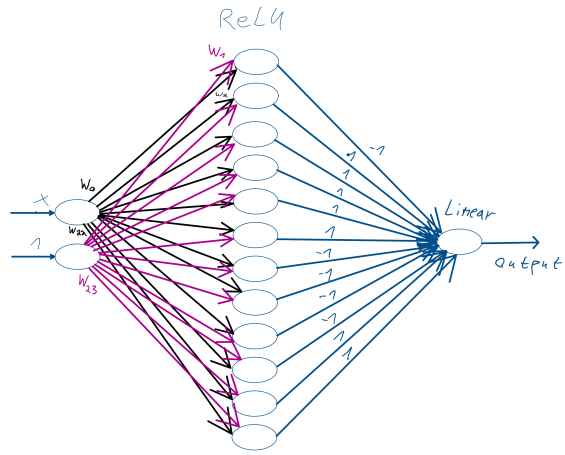
Figure 8a shows that the original polynomial function can be well approximated using just 12 weighted ReLU functions. The method starts by constructing linear approximations at specific points along $f(x)$, resulting in a piecewise linear approximation. Each segment is represented by a ReLU function, expressing the positive part of a linear combination of corresponding weights and biases, as illustrated in Fig. 8b. This enables the construction of a neural network which implements this ReLU-based approximation (as shown in figure 8c).



(a) Original function (blue) with its approximation using ReLU-based piecewise linear functions (pink)



(b) Illustration of Approximating the function f(x) using only weighted ReLU functions



(c) Illustration of the resulting Neural Network structure when using only weighted ReLU functions

Figure 8: ReLU function approximation and corresponding neural network architecture

In this architecture, the input x is scaled by weights, and biases are added, then each linear expression is processed by the ReLU activation function in the hidden layer. The outputs of the ReLU functions are then fed into the final output node, each weighted by either 1 or -1, depending on whether the corresponding hidden layer node represents values above or below zero. Because the output is linear, the resulting output is a composite of the individual weighted ReLU functions, effectively approximating the original target function. Increasing the number of hidden layer nodes would further improve the accuracy of this approximation, as each ReLU function would then cover a smaller segment of the function, refining the regression output. This improvement is evident in Section 2.4.1, where the comparison between the 1-3-1 ReLU model and the significantly larger 1-64-1 ReLU model demonstrates the superior performance of the larger network.

3 Classification

Some notations in the below section differs from Section 2, since they were assigned to different team members. The differences are clarified here to keep each section's report and corresponding code consistent. For subsection 2, layer indexing begins with 1, whereas this sections begins with 0 (input layer). Weighted input below is expressed as z as opposed to v, and output for layer L node i is expressed as $x^L_{\text{out},i}$ in Subsection 2, as opposed to x^L_i (with layer indexing differences, as stated above).

The IrisData.txt file contains 4 float value features for each instance, and contain 3 classes. For pre-processing, the class column y is first converted to unique class integers, and instances are split 7 : 3 for train and test sets with class balance to lessen bias. To ensure numeric stability, feature x in both sets are standardised using training set metrics, and y labels are one-hot-encoded just before the loss calculation. All random processes (weight initialisation and train-test-splits) use a fixed integer seed that increments with each repeated experiment, the results of which are averaged. This ensures reproducibility and helps distinguish genuine effects of the configuration from random fluctuations during training.

3.1 Task A: Deriving the Backpropagation Algorithm

3.1.1 Forward Pass

To obtain the backpropagation algorithm for a 4-5-3-3 classification network, the forward pass for the **first node of each of the 4 layers** is first outlined. Since the dataset has 4 features, the 4 nodes in the **input layer** $L = 0$ each correspond to a feature. The weighted input to node 1 of the first hidden layer (z^1_1) is therefore expressed as the weighted sum of features, and its activation (ϕ^1_1) using the *tanh* function (x^1_1, output of the node) are therefore:

$$z^1_1 = w^1_{1,1}x^0_1 + w^1_{2,1}x^0_2 + w^1_{3,1}x^0_3 + w^1_{4,1}x^0_4 + b^1_1, \qquad x^1_1 = \phi^1_1 = tanh(z^1_1) \tag{14}$$

where $w^L_{i,j}$ denotes the weight connecting output x^{L-1}_i of the i-th node from the previous layer $L-1$ to the j-th node of current layer L. b^L_j denotes the bias term to the weighted input z^L_j. We will use i, j, and L indices to denote these terms for sections in this task. Likewise for node 1 of layer 2 and node 1 of the output layer, the weighted inputs (z^2_1, z^3_1) and their activation (ϕ^2_1, ϕ^3_1) on the 5 outputs of layer 1 (x^1_i) and the 3 outputs of layer 2 (x^2_i) are as follows:

$$z^2_1 = w^2_{1,1}x^1_1 + w^2_{2,1}x^1_2 + w^2_{3,1}x^1_3 + w^2_{4,1}x^1_4 + w^2_{5,1}x^1_5 + b^2_1, \qquad x^2_1 = \phi^2_1 = tanh(z^2_1) \tag{15}$$

$$z^3_1 = w^3_{1,1}x^2_1 + w^3_{2,1}x^2_2 + w^3_{3,1}x^2_3 + b^3_1, \qquad x^3_1 = \phi^3_1 = \frac{e^{z^3_1}}{z^3_1, + z^3_2 + z^3_3} \tag{16}$$

The Softmax activation $x_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ is applied to the weighted inputs (z^3_i) of the output layer to normalise output x^3_1 as the probability of the first class, ensuring all 3 outputs (x^3_i) sum to 1. In addition to promoting training stability, the output values are directly interpretable as the model's predicted probability distribution over the classes.

3.1.2 Backward Pass

As above, the backward pass is illustrated with the first node of each of the 4 layers. The loss for this multi-class classification is given by the Cross Entropy Loss:

$$CE = -\sum_{j=1}^{3} d_j log(x^3_j), \qquad \text{where} \begin{cases} d_j = 1, \text{if true label is class } j \\ d_j = 0, \text{otherwise} \end{cases} \tag{17}$$

Using stochastic gradient descent (SGD), each update of the weight connecting the i-th node in layer $L-1$ to the j-th node in layer L can be expressed as:

$$w^L_{i,j,\text{new}} = w^L_{i,j,\text{old}} - \eta \frac{\partial CE}{\partial w^L_{i,j}} = w^L_{i,j,\text{old}} - \eta \frac{\partial CE}{\partial z^L_j} \frac{\partial z^L_j}{\partial w^L_{i,j}} \tag{18}$$

that is, the old weight minus the learning rate times the partial derivative of the loss function with respect to that weight. Similarly, the bias update for the j-th node in layer L can be expressed as:

$$b^L_{j,\text{new}} = b^L_{j,\text{old}} - \eta \frac{\partial CE}{\partial b^L_j} = b^L_{j,\text{old}} - \eta \frac{\partial CE}{\partial z^L_j} \frac{\partial z^L_j}{\partial b^L_j} = b^L_{j,\text{old}} - \eta \frac{\partial CE}{\partial z^L_j} \tag{19}$$

Using the chain rule, one may observe with (19) that $\frac{\partial z^L_j}{\partial b^L_j} = 1$, since b^L_j is constant. Hence, all weights and bias updates for node j in layer L share the term $\frac{\partial CE}{\partial z^L_j}$. Therefore, the backpropagation step can be further simplified to two terms: *nabla* ∇ and *delta* δ, where the updates for a node is:

$$w^L_{i,j,\text{new}} = w^L_{i,j,\text{old}} - \eta \nabla^L_{i,j}, \quad b^L_{j,\text{new}} = b^L_{j,\text{old}} - \eta \delta^L_j, \quad \nabla^L_{i,j} = \delta^L_j \frac{\partial z^L_j}{\partial w^L_{i,j}} \quad \delta^L_j = \frac{\partial CE}{\partial z^L_j} \tag{20}$$

Starting from layer 3 node 1, for CE loss (17) and softmax activation, we can compute:

$$\delta^3_1 = \frac{\partial CE}{\partial z^3_1} = \frac{\partial CE}{\partial x^3_1} \frac{\partial x^3_1}{\partial z^3_1} = x^3_1 - d_1, \quad \nabla^3_{i,1} = \delta^3_1 \frac{\partial z^3_1}{\partial w^3_{i,1}} = \delta^3_1 x^2_i \tag{21}$$

since $\frac{\partial z^3_1}{\partial w^3_{i,1}} = x^2_i$ by (16). Therefore, with $w^3_{i,1,\text{new}}$ repeated for every node i in layer 2, the weights and bias updates steps for layer 3 node 1 are:

$$w^3_{i,1,\text{new}} = w^3_{i,1,\text{old}} - \eta \cdot \delta^3_1 x^2_i, \quad b^3_{1,\text{new}} = b^3_{1,\text{old}} - \eta \cdot \delta^3_1, \quad \textbf{where} \quad \delta^3_1 = x^3_1 - d_1 \tag{22}$$

For layer 2 node 1, we have: $\delta^2_1 = \frac{\partial CE}{\partial z^2_1} = \frac{\partial CE}{\partial z^3_1} \frac{\partial z^3_1}{\partial x^2_1} \frac{\partial x^2_1}{\partial z^2_1} = \delta^3_1 \frac{\partial z^3_1}{\partial x^2_1} \frac{\partial (tanh(z^2_1))}{\partial z^2_1}$ by substituting x^2_1 from (15), and reusing δ^3_1 from (21). Therefore, $\delta^2_1 = \delta^3_1 \cdot w^3_{1,1} \cdot (1 - tanh^2(z^2_1))$, by deriving $\frac{\partial z^3_1}{\partial x^2_1}$ from (16). As above, $\nabla^2_{i,1} = \delta^2_1 \frac{\partial z^2_1}{\partial w^2_{i,1}} = \delta^2_1 \cdot x^1_i$, repeating $w^2_{i,j}$ for every node i in layer 1. Similarly, since it also uses *tanh* activation, we have for layer 1 node 1: $\delta^1_1 = \delta^2_1 \cdot w^2_{1,1} \cdot (1 - tanh^2(z^1_1))$ and $\nabla^1_{i,1} = \delta^1_1 \cdot x^0_i$, again repeating $\nabla^1_{i,1}$ with $w^1_{i,j}$ for every node i in layer 0 (features). Applying these gradient calculations, the weights and bias update steps for node 1 of layer 2 and 1 can be generalised to:

$$w^L_{i,1,\text{new}} = w^L_{i,1,\text{old}} - \eta \cdot \delta^L_1 \cdot x^{L-1}_i, \quad b^L_{1,\text{new}} = b^L_{1,\text{old}} - \eta \cdot \delta^L_1, \quad \textbf{where} \quad \delta^L_1 = (\delta^{L+1}_1 w^{L+1}_{i,1} \cdot (1 - tanh^2(z^L_1)) \tag{23}$$

To generalise to node j every layer L, the calculations in (22) and (23) are repeated for every j-node, each repeated for every i-node in layer $L-1$.

3.2 Task B: Training, and testing with stochastic gradient descent

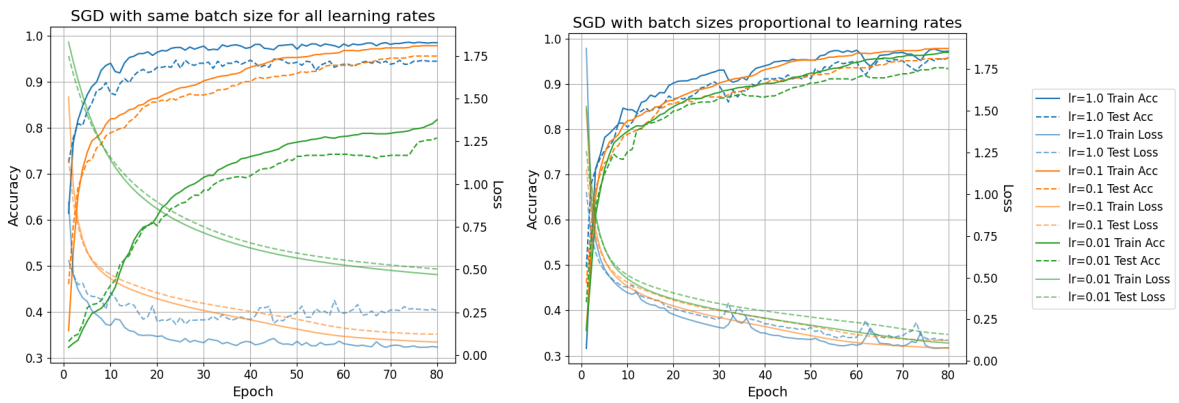
3.2.1 Generalisation to Matrix Notation and SGD Implementation

Since (20) iterates over i for every node j in layer L, the network can be implemented as matrices and vectors. For layers 1 to 3, each layer's transformation is represented by a weight

matrix w^L and a *nabla* matrix ∇^L of shape $i \times j$, storing the weights and its gradients. The bias b^L, weighted input z^L, activation (output) x^L, and *delta* δ^L for layer L are expressed as vectors of length j, where δ^L is the gradients to b^L.

In the forward pass, the weighted input for layers 1 to 3 is $z^L = w^L x^{L-1} + b^L$, with activation $x^L = tanh(z^L)$ for layers 1 and 2, and $x_i^3 = softmax(z_i^3) = \frac{e^{z_i^3}}{z_1^3, + z_2^3 + z_3^3}$ for layer 3. In the backward pass, we first obtain the loss by (17) on outputs x_j^3 and labels d_j. From layer 3 to 1, we obtain δ^L (bias gradient) then ∇^L (weight gradients), iterating L backwards. Starting from layer 3, we have $\delta^3 = x^3 - d$ and $\nabla^3 = \delta^3 \cdot x^3$ from (22). Then, by (23), for layer 2: $\delta^2 = \delta^3 \cdot w^3 \cdot (1 - tanh^2(z^2))$ and $\nabla^2 = \delta^2 \cdot x^2$. Similarly, for layer 1: $\delta^1 = \delta^2 \cdot w^2 \cdot (1 - tanh^2(z^1))$ and $\nabla^1 = \delta^1 \cdot x^1$. For smoother descent, using SGD as outlined in (22) and (23), each layer's δ^L and ∇^L are **accumulated by batch**, then subtracted from w^L and b^L after applying learning rate η / batch size.

3.2.2 SGD Results and Discussion



(a) SGD with batch size 16 for all learning rates

(b) SGD with batch sizes 160, 16, 2, proportional to learning rates 1.0, 0.1, 0.01

Figure 9: Comparison of SGD training dynamics for accuracy and loss (averaged over 10 runs) with (a) fixed batch size (16) and (b) batch sizes proportional to learning rates.

Figure 9a shows the average accuracy and loss over 10 training runs for all learning rates, using a uniform batch size of 16. In this configuration, the rate of convergence decreases with the learning rate, with the highest learning rate (1.0) reaching an accuracy of 0.8 before epoch 5, while learning rates 0.1 and 0.01 achieve the same accuracy around epoch 10 and 80 respectively. Furthermore, training runs using learning rates 1.0 and 0.1 both achieved a test accuracy of approximately 0.95, while the 0.01 learning rate run converged more gradually and achieved a final test accuracy closer to 0.8 at epoch 80. Although using 1.0 as the learning rate converges the quickest, the loss over epochs reveals higher volatility relative to the smoother descent of the training runs using lower learning rates, demonstrating the trade-off between faster convergence and training stability.

To achieve similar convergence rates, Figure 9b shows the results when batch sizes are scaled proportionally to learning rates. More specifically, batch sizes 160, 16, and 2 (rounded) correspond to learning rates 1.0, 0.1, 0.01, scaled up or down from batch size of 16 in Figure 9a. The volatility on the highest learning rate (1.0) persists even when the batch size is scaled proportionally, as seen by the three loss spikes in Figure 9b. Notably, since the batch size (160) exceeds the number of training examples (105), the algorithm effectively performs full-batch gradient descent - updating once per epoch on the average gradient over all training examples.

In theory, this makes the updates more deterministic than mini-batch updates, but resulted in noticeable spikes in epoch loss even when averaged over 10 runs, indicating overshoots of local minima from single large update steps in the parameter space. However, the momentary loss spikes do not necessarily suggest poor choice of step size, as they are distinct from persisting oscillations that indicate failure to converge, with all three configurations achieving similar convergence rate and final test accuracy. Despite this, pairing smaller learning rates with smaller batch sizes may still be preferred, as they provide more stable training dynamics and avoid steep drops in performance from overly aggressive updates.

3.3 Task C: Training and Testing with other Optimisers

To compare the training dynamics of SGD against momentum-based and adaptive optimisers, the gradient update procedure is modified for **SGD with momentum** and **Adam**. Both implementations retain identical network initialisation, forward, and backward passes as SGD, with each optimiser updating gradients on additional terms calculated with ∇^L (weight gradients) and δ^L (bias gradients) outputted from the backwards pass.

3.3.1 Implementing SGD with Momentum

For SGD with momentum, instead of directly applying bias and weight gradients (∇^L and δ^L) like SGD, the velocity terms v_w^L and v_b^L are introduced. The velocity terms are initialised to 0 with the same shape as w^L and b^L (matching ∇^L and δ^L), so that each parameter has a corresponding velocity. After the backwards pass, the velocities are first calculated and tuned with the momentum coefficient ρ, then applied to the parameters updates as follows:

$$v_{w,\text{new}}^L = \rho \cdot v_{w,\text{old}}^L + \nabla^L, \quad v_{b,\text{new}}^L = \rho \cdot v_{b,\text{old}}^L + \delta^L, \quad w_{\text{new}}^L = w_{\text{old}}^L - \eta \cdot v_{w,\text{new}}^L, \quad b_{\text{new}}^L = b_{\text{old}}^L - \eta \cdot v_{b,\text{new}}^L \tag{24}$$

We set the ρ to 0.9, accumulating and averaging velocity by batch before subtraction as SGD.

3.3.2 Implementing the Adam Optimiser

Similarly, the Adam optimiser tracks the first moment (m_w^L and m_b^L) and second moment (v_w^L and v_b^L), approximating the running average of the gradients and their squares. Each matrix is initialised to zero with the same shape as their corresponding weight (w^L) or bias (b^L) matrices. Then, the weights' first (m_w^L) and second moment (v_w^L) are updated as:

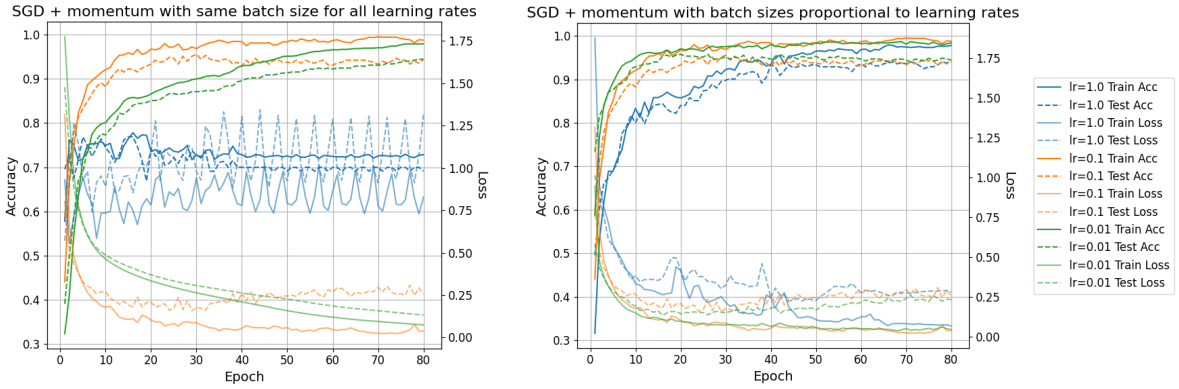
$$m_{w,\text{new}}^L = \beta_1 \cdot m_{w,\text{old}}^L + (1-\beta_1)\nabla^L \qquad v_{w,\text{new}}^L = \beta_2 \cdot v_{w,\text{old}}^L + (1-\beta_2)(\nabla^L)^2 \tag{25}$$

where ∇^L is the weight gradient and $\beta_1, \beta_2 \in (0,1)$ are the exponential decay rates. To mitigate slow convergence from small running averages on initialisation, bias correction is applied, where t effectively denotes the batch number (time step). Finally, the parameters are updated:

$$\text{Correction}: \hat{m}_{w,\text{new}}^L = \frac{m_{w,\text{old}}^L}{1-(\beta_1)^t}, \quad \hat{v}_{w,\text{new}}^L = \frac{v_{w,\text{old}}^L}{1-(\beta_2)^t}, \quad \text{Update}: w_{\text{new}}^L = w_{\text{old}}^L - \eta \frac{\hat{m}_{w,\text{new}}^L}{\sqrt{\hat{v}_{w,\text{new}}^L} + \epsilon} \tag{26}$$

where ϵ is a small number to avoid division-by-zero issues. Dividing by $\sqrt{\hat{v}_{w,\text{new}}^L}$ scales down repeated large gradients and scales up repeated small gradients, effectively accelerating and stabilising parameter-wise convergence. In all experiments, β_1, β_2, and ϵ are set to 0.9, 0.999, and 10^{-8}, respectively, and the above process is repeated for the biases by substituting ∇^L with δ^L. Keeping consistent with the other optimisers, the update terms with η are accumulated and averaged by batch.

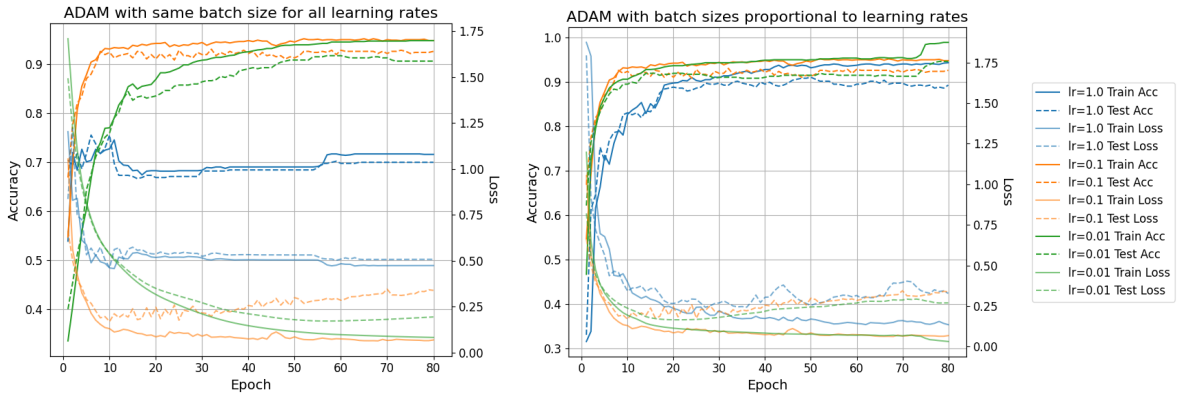
3.3.3 Results and Comparison to Vanilla SGD



(a) SGD with Momentum (batch size 16 for all LRs)

(b) SGD with Momentum (batch sizes 160, 16, 2, proportional to LRs)

Figure 10: Comparison of SGD with Momentum training dynamics for accuracy and loss (averaged over 10 runs) with (a) fixed batch size (16) and (b) batch sizes proportional to learning rates 1.0, 0.1, 0.01.



(a) Adam (batch size 16 for all LRs)

(b) Adam (batch sizes 160, 16, 2, proportional to LRs)

Figure 11: Comparison of Adam training dynamics for accuracy and loss (averaged over 10 runs) with (a) fixed batch size (16) and (b) batch sizes proportional to learning rates 1.0, 0.1, 0.01.

In principle, adding a velocity term in **SGD with momentum** should provide **faster convergence**. Comparing the training dynamics in Figure 10a to SGD in Figure 9a, both learning rates 0.1 (orange) and 0.01 (green) converge faster, achieving test accuracies of 0.9 and 0.8 at epoch 10, as opposed to 0.8 and 0.45 when using SGD. In fact, by adding the momentum term, the rate of convergence in both accuracy and loss are comparable to using an order of magnitude's higher learning rate with SGD. However, momentum in itself does not stabilise training, as highlighted by the **large oscillations** and **suboptimal convergence** when using learning rate of 1.0 (blue) in Figure 10a. By accumulating most of the past gradients (since $\rho = 0.9$), all update steps **overshoot the local minimas** and achieves test accuracy of 0.7. Notably, this indicates suboptimal convergence but not complete convergence failure, since the average test accuracy over 10 runs remains higher than random prediction (0.33). Since this does not occur when using vanilla SGD ($\rho = 0$), we can hypothesise that using lower ρ may better balance stability with convergence rate.

The training instability using momentum persists when batch sizes are scaled proportionally to the learning rate. This can be seen with the loss progression using learning rate 1.0 (blue) in Figure 10b showing more unpredictable and frequent spikes than its counterpart in SGD (Figure 9b). Furthermore, when using learning rates 0.1 and 0.01, the train and test accuracies diverge earlier and greater than their SGD counterparts, suggesting the possibility of **overfitting when updating more aggressively** on the training set.

Figure 11a shows the loss progression using the Adam optimiser, when all learning rates use the same batch size. Overall, the training dynamics between Adam and SGD with momentum are comparable, except two points. Firstly, Adam **displayed faster convergence in the initial warm up** than SGD with momentum. With regards to the test accuracy when using learning rate of 0.1 (orange), Adam reaches 0.93 accuracy 10 epochs earlier than SGD with momentum, largely due to the **bias correction** step scaling up the momentum and velocity during the initial training steps when the running average is near zero. The training loss also **plateaus earlier** than SGD with momentum when using learning rate of 0.1 (green), as the momentum is scaled up in the near minimas, even though it did not translate to better performance. Secondly, the Adam's training dynamics are **more stable** than SGD with momentum, as shown by the absence of large oscillations observed when using learning rate of 1.0 in Figure 10a, and the **reduced oscillation** in the later epochs when using 0.1 learning rate. This suggests Adam is **more capable at mitigating minima overshoots** than SGD with momentum, dampening large swings using adaptive updates. However, the reduced swings may have resulted in **premature convergence**, as the test accuracy of 0.5 using learning rate of 1.0 (blue) is lower than the 0.7 achieved using SGD with momentum.

The dampening of large swings when using adaptive updates is also observed when batch size is scaled proportionally (Figure 11b), as the loss and accuracy progression are much smoother than those in Figure 10b. However, despite providing a higher rate of convergence, Adam is unable to optimise beyond test accuracy of 0.9 when using batch size of 160 (1 update per epoch) when using learning rate of 1.0 (blue), unlike SGD with momentum. This suggests the lack of regularisation effect with stochastic gradient noise when updating infrequently causes Adam to **prematurely settle in suboptimal minimas**, that is less pronounced the other optimisers. This is likely due to Adam's bias correction step scaling up small initial gradients, leading to **insufficient early exploration of the parameter space**.

3.3.4 Conclusion on Optimiser Experiments

Despite all three optimisers achiveing similar final test accuracies, they differ in their path to convergence. SGD's is more forgiving to batch size and learning rate mismatches, although the rate of convergence on smaller learning rates requires vastly more number of epochs. By adding the momentum term, the rate of convergence is comparable to using 10 times higher learning rate without momentum, but is less forgiving to higher learning rates in training stability. Adam warms up and stablises more quickly due to bias correction and adaptive step sizes, but may settle into suboptimal minimas due to reduced stochastic noise from infrequent updates and over-aggressive early updates.

3.4 Task D: Exploring different Network Configurations using TensorFlow

3.4.1 Network Architectures and Training Setup

To explore different neural network architectures and activation functions for Iris flower classification, we tested five different configurations using TensorFlow, all using SGD. We evaluated the original architecture (4-5-3-3) with tanh activation as our baseline, a smaller network (4-3-3) to test minimal complexity, a larger architecture (4-8-6-3) to examine the

impact of additional nodes, a ReLU variant of our original architecture to compare activation functions, and a deeper network (4-6-4-4-3) to investigate the effect of additional layers.

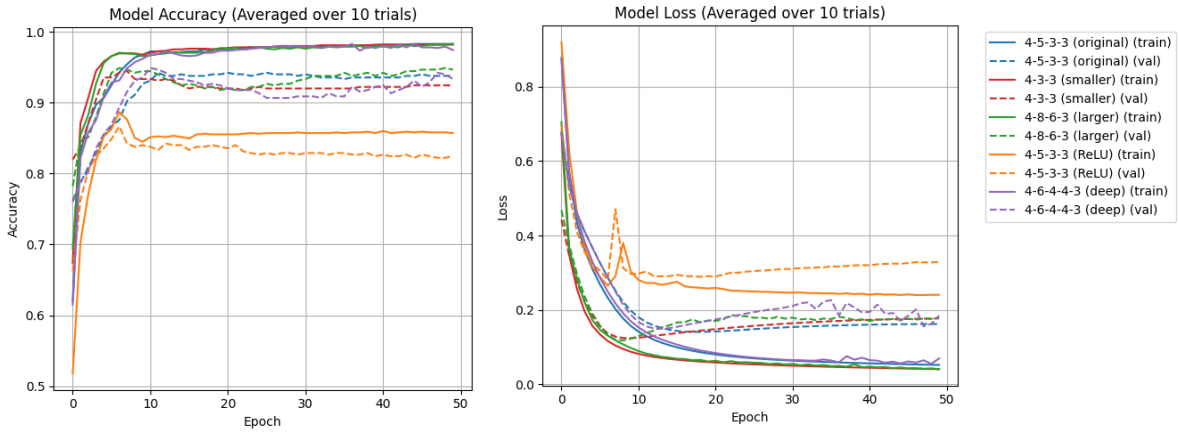
To ensure fair comparison, we first conducted a hyperparameter search to find optimal learning rates for each architecture. The search was performed with a step size of 0.01, testing learning rates from 0.01 to 1.80, with 10 training runs per learning rate using different random seeds. The optimal rates for each architecture are shown in Table 3.

Architecture	**4-5-3-3 (original)**	**4-3-3 (smaller)**	**4-8-6-3 (larger)**	**4-5-3-3 (ReLU)**	**4-6-4-4-3 (deep)**
Best LR	0.35	1.23	0.60	0.85	0.27

Table 3: Optimal learning rates for each architecture variant.

3.4.2 Experiment Results for Network Architecture Comparison

Following hyperparameter tuning, each architecture was trained 10 times with different random seeds to assess stability. The training process yielded the accuracy and loss curves shown in Figure 12, with final performance metrics summarised in Table 4.



(a) Training and Validation Accuracy (b) Training and Validation Loss

Figure 12: Training curves showing accuracy and loss progression

Architecture	**Train Acc.**	**Val. Acc.**	**Train Loss**	**Val. Loss**
4-5-3-3 (original)	**0.9829 ± 0.0057**	0.9378 ± 0.0218	0.0523 ± 0.0043	**0.1624 ± 0.0341**
4-3-3 (smaller)	0.9819 ± 0.0029	0.9244 ± 0.0147	**0.0410 ± 0.0087**	0.1774 ± 0.0235
4-8-6-3 (larger)	**0.9829 ± 0.0057**	**0.9467 ± 0.0147**	0.0408 ± 0.0064	0.1786 ± 0.0262
4-5-3-3 (ReLU)	0.8571 ± 0.2374	0.8244 ± 0.1969	0.2408 ± 0.3813	0.3291 ± 0.3198
4-6-4-4-3 (deep)	0.9743 ± 0.0113	0.9333 ± 0.0263	0.0698 ± 0.0285	0.1849 ± 0.0543

Table 4: Performance comparison across architectures (mean ± std over 10 trials)

3.4.3 Analysis for Network Architecture Comparison

Table 4 shows the largest architecture (green) having the strongest overall performance in validation accuracy (94.67% ± 1.47%) while maintaining excellent generalisation capabilities. The performance gap between training (98.29% ± 0.57%) and validation accuracy is the smallest of all configurations, suggesting that the additional width improved the model's ability to capture relevant features without overfitting. It also shows **stabler performance across seeds**, evident from the lower standard deviation in performance across repeated runs,

indicating an appropriate model capacity relative to the dataset size and complexity. Interestingly, the smaller network (4-3-3, red) performed surprisingly well, achieving the lowest training loss (0.0410 ± 0.0087) while maintaining competitive validation accuracy (92.44% ± 1.47%). This indicates a **reduced architecture could effectively capture the discriminating patterns** for classification, suggesting that the Iris dataset's feature space is not particularly high dimensional and have linear or **low dimensional decision boundaries**.

Around epoch 5, the ReLU variant (orange) appeared to experience a simultaneous spike in both accuracy and loss, which may occur when cross entropy loss imposes high penalties for overconfident but incorrect predictions without impacting accuracy. However, the more plausible explanation is the accuracy improved at a rate normal for early epochs, only experiencing a sharp downturn *after* the loss increase (i.e. not a spike in accuracy). This can likely be attributed to the **dying ReLU problem**, where neurons become permanently inactive from zero gradients arising from negative inputs. This **prevents gradient flow** through those neurons, impacting the model's learning capacity. Consequently, the ReLU variant significantly underperformed compared to its tanh counterpart, achieving only 82.44% ± 19.69% validation accuracy with **notably high variability** across repeated runs. The bounded and centered nature of tanh (-1 to 1) appears to provide stabler gradients than ReLU's unbounded positive range for this small dataset, particularly given the simple decision boundaries needed for Iris classification. The high variability across runs also suggest that **weight initialisation is be critical** to the frequency of neuron deaths when using ReLU, which can be mitigated by using alternative activations such as Leaky ReLU or ELU to avoid a gradient dead zone near zero, without having tanh's vanishing gradients property near zero and one.

The deeper network (4-6-4-4-3) achieved solid performance with 93.33% ± 2.63% validation accuracy, but didn't outperform the original and wider architecture - despite its increased complexity. While showing competitive accuracy, it had slightly higher loss values (training: 0.0698 ± 0.0285, validation: 0.1849 ± 0.0543) compared to other architectures, indicating the additional depth had not translated to better generalisation. This means the **additional capacity had gone unused**, likely caused by the Iris datasets low dimensional feature space. Furthermore, when compared to its counterparts, the deeper network (purple) showed **greater rate of divergence between test and train metrics** around epoch 10, before slightly recovering onwards of epoch 30 with jagged progression. This highlights higher-capacity model's tendency to **overfit on small datasets**, as well as the more challenging and volatile optimisation with greater depth, that may be mitigated methods such as batch normalisation. Like the ReLU variant, the deeper model also shows higher standard deviation over repeats, indicating the necessity for more careful weight initialisation as model depth increases.

3.4.4 Conclusion for Network Architecture Comparison

For Iris classification, wider networks proved more capable than deeper models. The increased model depth made optimisation less stable, and did not translate to better generalisation. The experiments also highlighted the impact of activation functions on the training dynamics, with the tanh function providing better and stabler optimisation over ReLU. The surprisingly strong performance of the smaller architecture suggests that the Iris dataset's classes are easily separable or have low dimensional features, thus requiring less capacity to capture the critical decision boundaries, though the wider network provided modest improvements in accuracy and stability. These results emphasise that careful consideration of activation functions, model capacity relative to dataset size and complexity, and careful optimisation techniques are more crucial than simply increasing network complexity.

4 Transfer Learning

4.1 Task A: Dataset Preparation

The dataset consists of five different fruit classes: durian, papaya, kiwi fruits, mangosteen, and mango. For each class, 15 images were collected, resulting in a total dataset of 75 images. Each image was preprocessed to meet the required specifications of 227 × 227 × 3 pixels in uint8 format.

(a) Durian (b) Kiwi (c) Mango (d) Mangosteen (e) Papaya

Figure 13: Sample images from each fruit class after preprocessing to 227 × 227 × 3 pixels

4.2 Task B and C: Transfer Learning Implementation and Results

4.2.1 Network Architecture and Modifications

Both AlexNet and GoogLeNet were adapted for our five-class fruit classification task through transfer learning. For both networks, we preserved the original convolutional layers to use their pre-trained feature extractors, while modifying the final fully connected layer from 1000 to 5 output nodes to match our fruit classes. GoogLeNet retained its auxiliary classifiers to combat vanishing gradients, while AlexNet maintained its original ReLU activations and dropout layers to ensure effective feature learning and prevent overfitting.

4.2.2 Training Configuration and Performance

Table 5: Network Training Configuration

Parameter/Metric	Optimiser	Learning Rate	Batch Size
AlexNet	SGD (momentum 0.9)	0.001	32
GoogLeNet	SGD (momentum 0.9)	0.01	32

Table 6: Key Training Milestones

Network	**Epoch**	**Training Loss**	**Validation Loss**	**Validation Accuracy**
AlexNet	1	1.5558	1.3137	39.13%
	4	0.2010	0.7670	73.91%
	10	0.0050	0.9047	73.91%
GoogLeNet	1	1.6392	1.5639	39.13%
	7	0.2150	0.6679	91.30%
	10	0.0423	0.4919	86.96%

Both networks showed consistent improvement during training, but with notable differences in performance dynamics. AlexNet reached a stable validation accuracy of 73.91% by epoch 7, which was maintained consistently thereafter. In contrast, GoogLeNet demonstrated superior

learning capability, achieving a peak validation accuracy of 91.30% at epoch 7, before settling at 86.96% in the final epochs.

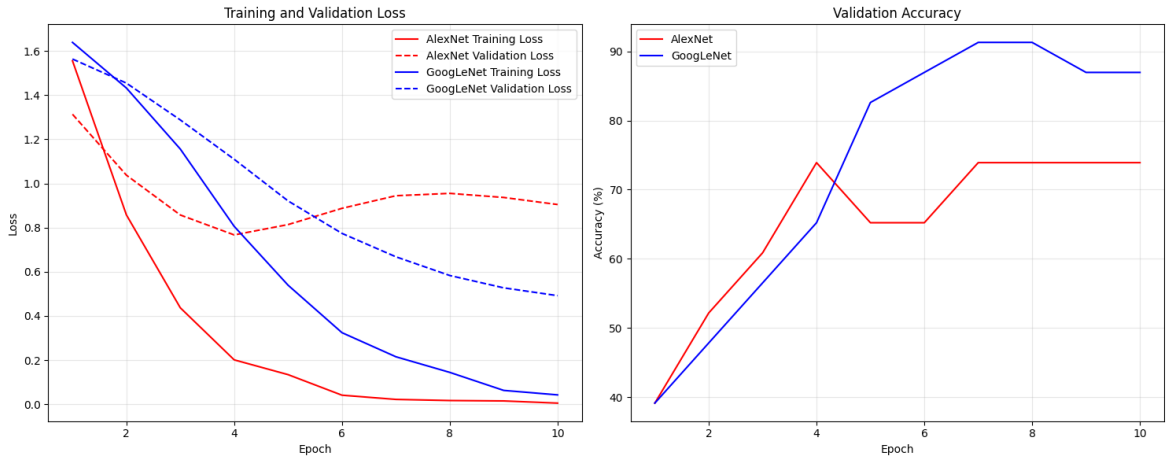


Figure 14: Training dynamics comparison between AlexNet (red) and GoogLeNet (blue). The left plot shows training and validation losses (solid and dashed lines respectively), while the right plot shows validation accuracy. GoogLeNet demonstrates faster convergence and achieves higher peak accuracy compared to AlexNet.

4.2.3 Classification Analysis

(a) AlexNet (b) GoogLeNet

Figure 15: Comparison of confusion matrices between networks. GoogLeNet achieves perfect classification for durian, mango, and papaya, while AlexNet shows strongest performance with mangosteen and durian.

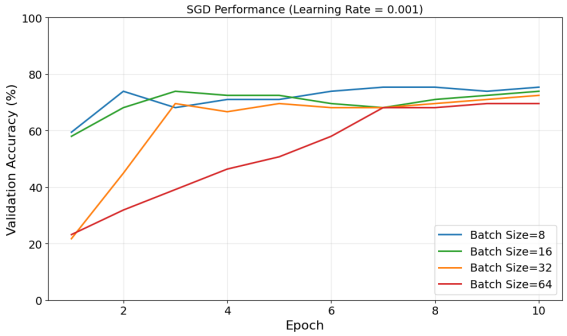
The networks demonstrated distinct classification patterns. GoogLeNet achieved perfect classification for durian, mango, and papaya classes, while AlexNet showed particular strength with mangosteen and durian identification. Both networks experienced some confusion with kiwi fruits, though GoogLeNet handled this challenge more effectively. AlexNet exhibited more pronounced confusion between mango and papaya pairs, attributable to their similar colour patterns and shapes in certain orientations.

4.2.4 Discussion

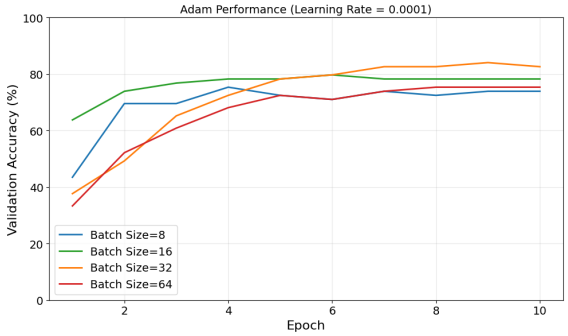
This analysis demonstrates GoogLeNet's architectural superiority over AlexNet, with its inception modules enabling better feature extraction and significantly higher accuracy (91.30% vs 73.91%) in the fruit classification task. The training dynamics showed GoogLeNet achieved faster convergence despite using a higher learning rate (0.01 vs 0.001), suggesting more robust gradient flow through its auxiliary classifiers. While both networks showed good generalisation given the limited dataset size, GoogLeNet demonstrated better robustness to visual similarities between fruit classes. The slight decrease in GoogLeNet's validation accuracy in the final epochs suggests that early stopping might be beneficial to prevent overfitting, though its final performance remained substantially above that of AlexNet. These results highlight the advantages of GoogLeNet's more sophisticated architecture in transfer learning applications, particularly for fine-grained visual classification tasks with limited training data.

4.3 Task D: Hyperparameter Optimisation Analysis

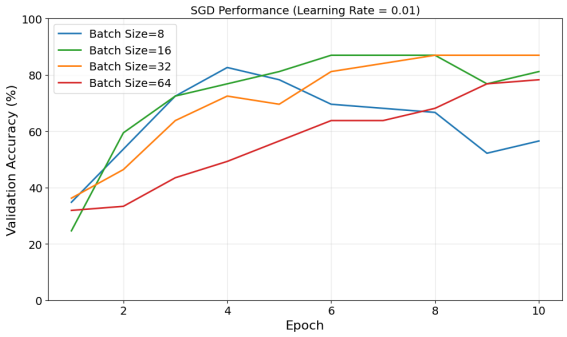
4.3.1 Optimiser Performance Analysis



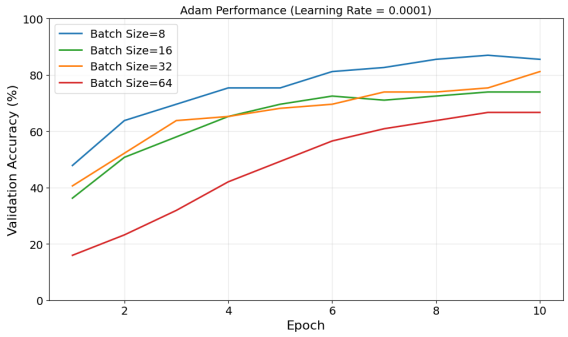
(a) AlexNet SGD Performance



(b) AlexNet Adam Performance



(c) GoogLeNet SGD Performance



(d) GoogLeNet Adam Performance

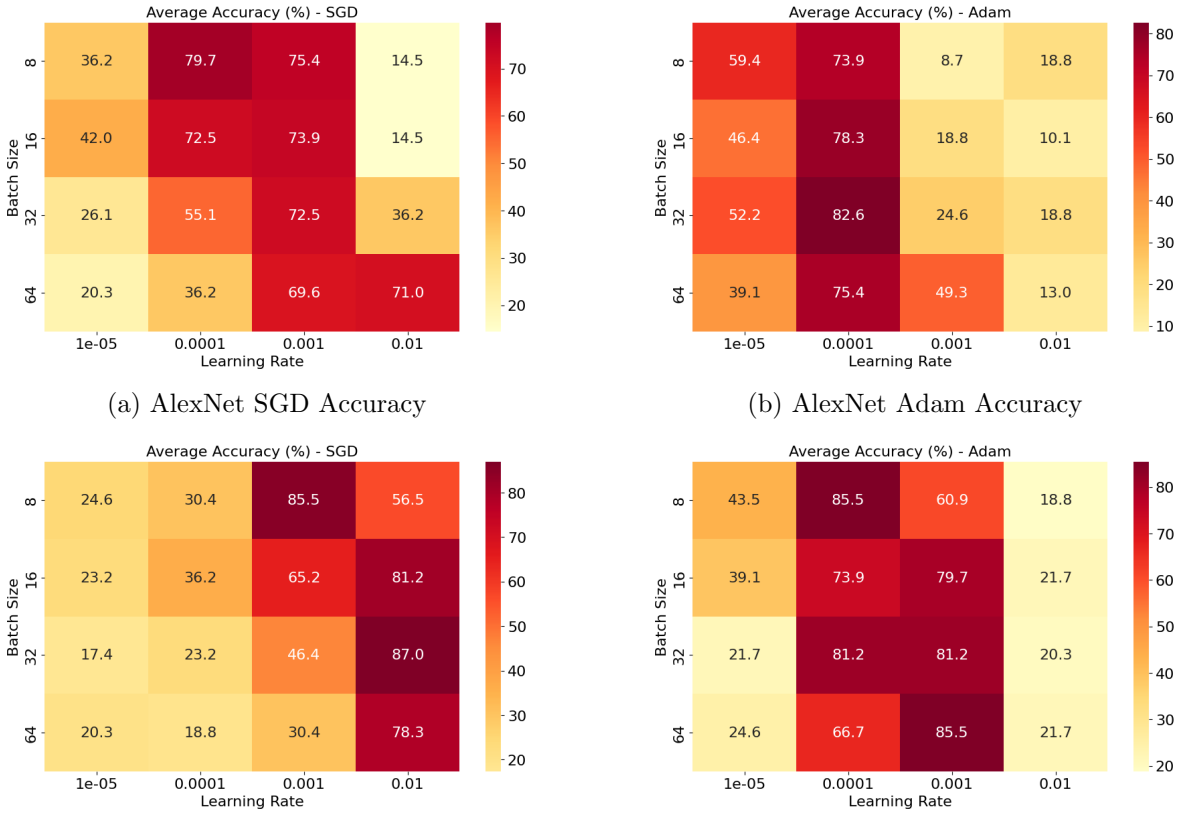
Figure 16: Performance comparison of AlexNet and GoogLeNet with SGD and Adam optimisers at their optimal learning rates. The plots show validation accuracy over epochs for different batch sizes.

AlexNet's performance varied between optimisers. With SGD at learning rate 0.001, batch size 8 showed fastest initial learning, reaching 70% accuracy by epoch 2, while batch size 8 started slightly slower but achieved similar performance by epoch 3. Larger batches (32, 64) showed gradual improvement, eventually reaching 70-75% accuracy by epoch 10. Using Adam at learning rate 0.0001, all batch sizes demonstrated more uniform convergence, starting at 40-50% and steadily improving to 70-80% accuracy, with remarkably consistent performance across different batch sizes.

GoogLeNet demonstrated different characteristics across optimizers. Under SGD at 0.01, smaller batches (8, 16) showed rapid initial learning reaching 70% by epoch 3, but exhibited higher variance throughout training. Larger batches had slower initial convergence but eventually achieved similar performance, with batch size 64 showing the most stable trajectory. With Adam at 0.0001, GoogLeNet showed smoother learning curves across all batch sizes, with slightly better final performance for batch size 8 reaching 85% accuracy, while maintaining consistent improvement with minimal fluctuations compared to SGD.

The performance characteristics notably differed between optimisers. SGD showed higher variance in learning trajectories but achieved quick initial learning with smaller batches, while Adam demonstrated more predictable and stable learning curves across all batch sizes, suggesting it might be the more reliable choice for GoogLeNet when training stability is prioritised.

4.3.2 Cross-Architecture Comparison



(a) AlexNet SGD Accuracy

(b) AlexNet Adam Accuracy

(c) GoogLeNet SGD Accuracy

(d) GoogLeNet Adam Accuracy

Figure 17: Accuracy heatmaps comparing hyperparameter sensitivity across architectures and optimisers. Darker colors indicate higher accuracy.

The heatmaps show GoogLeNet maintained consistent performance across hyperparameter combinations, particularly at middle to higher range learning rates and consistent regardless of the batch sizes. While AlexNet showed high sensitivity to optimiser choice and learning rates with distinct sweet spots, GoogLeNet exhibited more robust behaviour throughout the parameter space. This architectural advantage was especially prominent with Adam optimisation at lower learning rates, suggesting GoogLeNet's inception modules provide both improved performance and more reliable training dynamics, making it a more robust choice for transfer learning applications.

5 Q-Learning

In supervised learning, models map inputs to outputs using labeled data. Q-learning, by contrast, is label-free and learns through trial-and-error to maximise rewards, where a reward indicates performance in a given state. Initially, the agent explores random actions, later exploiting knowledge to optimise rewards.
We use a grid world (Fig. 18) to illustrate Q-learning, defining all possible states. The goal is to identify optimal actions for each state-action pair to maximise rewards.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 18: Grid World

To describe the environment and agent, we used the Markov Decision Process (MDP) model with elements (S, A, p, R, S').
S is the set of possible states, with S_t representing the environment's state at iteration t. A is the set of actions (up, down, left, right), with A_t being the action at iteration t. p is the transition function, describing the probability of moving from the current state to a new state s'. $R(s, a, s')$ is the reward function, giving the reward when action a transitions from state s to s'. A negative reward of -1 applies to every state transition to minimise energy required for the agent to reach the goal, this is called the "living reward."
Three states differ in Fig. 18. State 6 represents an obstacle, meaning the agent cannot enter it. Practically, obstacles could be areas a robot must avoid. State 12 gives a positive reward of $+10$, representing a target area the agent should reach. State 8 has a negative reward of -10, representing restricted areas or energy-draining situations. States 12 and 8 are terminal states, where the agent gets stuck after entering.
The probabilities $p(s'|s, a)$ and rewards $R(s, a, s')$ are not known a priori but are sampled through actions. Each sample provides an estimate of the ideal Q-value, combining the immediate reward and the discounted maximum Q-value of the next state:

$$Sample = R(s, a, s') + \gamma \max Q^*(s', a') \tag{27}$$

Q-values are refined using the Bellman Error, updated iteratively via:

$$Q_{i+1}(s, a) = Q_i(s, a) + \alpha \left(\underbrace{Sample - Q_i(s, a)}_{\text{Bellman Error}} \right) \tag{28}$$

The learning rate α balances sensitivity and stability; here, $\alpha = 0.1$ ensures convergence and robustness.
Exploration and exploitation are balanced using the ϵ-greedy method. The exploration probability ϵ decays over time, ensuring the agent continues discovering new paths. The decay formula follows MATLAB's Deep Q-Network (DQN) implementation:

$$\epsilon = \epsilon \cdot (1 - \text{EpsilonDecay}) \tag{29}$$

Initially, $\epsilon = 1$ with $EpsilonDecay = 0.005$, and a minimum $\epsilon_{min} = 0.01$ ensures a persistent exploration rate of 1%. Exploration occurs if a random number $a \in [0, 1]$ is less than ϵ; otherwise, the agent exploits known optimal actions.

The First Episode

With the algorithm defined, the Q-learning process can commence. To illustrate its application, the first three iterations of the initial three episodes are presented, beginning with the first episode. At the start of Q-learning, the Q-table is initialised with zeros, where each entry corresponds to a state-action pair, as can be seen in Fig. 7.

The 1st Iteration

Moreover, it is important that we start with the same environment for each episode. For this, we therefore chose state 1 as the starting environment.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 19: Grid World

As previously discussed, a random number a is generated and compared to the current ϵ value to determine whether the agent explores or exploits. In this instance, the current α (learning rate) is 0.223902, and ϵ (exploration probability) is 0.995000. Based on this comparison, the selected action is "$right$". At this stage, the Q-value for the state-action pair $Q(1, Right)$ is 0.
The agent subsequently transitions to cell 2, where all Q-values are initialised to zero, meaning the choice of action for maximising the Q-value is inconsequential. Using the update formulas presented earlier in Equations 27 and 28, and accounting for a living reward of -1, a learning rate (α) of 0.1, and a discount factor (γ) of 0.9, the Q-value is updated as follows:

$$Sample = R(1, right, 2) + \gamma \max Q^*(2, any) = -1 + 0.9 \cdot 0 = -1$$

$$Q_{i+1}(1, right) = Q_i(1, right) + 0.1(Sample - Q_i(1, right)) = 0 + 0.1(-1 - 0) = -0.1$$

Having chosen an action that does not end up in a rewarding state results in a negative Q-value (see table 8), reinforcing the intuitive idea that effort without reward should be penalised.

	Up	Right	Down	Left
Cell 1	0	0	0	0
Cell 2	0	0	0	0
Cell 3	0	0	0	0
Cell 4	0	0	0	0
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 7: Q-Table at the beginning of the first episode

	Up	Right	Down	Left
Cell 1	0	-0.1	0	0
Cell 2	0	0	0	0
Cell 3	0	0	0	0
Cell 4	0	0	0	0
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 8: Q-table after the first iteration

The 2nd Iteration

This leads to the next state, where the learning process is continued.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 20: New state of the agent

The same process is repeated at this step: In this instance, $a = 0.363644$ and $\epsilon = 0.990025$, resulting in the random selection of the action "*down*". At this point, the Q-value for the state-action pair $Q(2, Down)$ is 0.

However, as there is no valid cell to transition to from the current state by moving down, the agent remains in cell 2. In this state, no action yields a maximum Q-value, as all Q-values are zero. Applying the update formulas outlined in Equations 27 and 28, the Q-value is updated as follows:

$$Sample = R(2, down, 2) + \gamma \max Q^*(2, any) = -1 + 0.9 \cdot 0 = -1$$

$$Q_{i+1}(2, down) = Q_i(2, down) + 0.1(Sample - Q_i(2, down)) = 0 + 0.1(-1 - 0) = -0.1$$

The Q-table is then again updated using the new value for $Q(2, down)$.

	Up	**Right**	**Down**	**Left**
Cell 1	0	-0.1	0	0
Cell 2	0	0	-0.1	0
Cell 3	0	0	0	0
Cell 4	0	0	0	0
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 9: Updated Q-Table after second iteration

The 3rd Iteration

The agent has remained in cell 2 from the previous action.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 21: State of the agent in the beginning of iteration 3

In the third iteration, the random number a is generated as 0.783214, while the current value of ϵ is 0.985075. As $a < \epsilon$, a random action is selected once again, which in this case is "*down*". Consequently, the agent remains in cell 2.

At this stage, a Q-value already exists for the state-action pair $Q(2, Down) = -0.1$. Therefore, in cell 2, the actions that yield the maximum possible Q-value are all actions except "*down*". Applying the previously introduced formulas in Equations 27 and 28, the updated Q-value is computed as follows:

$$Sample = R(2, down, 2) + \gamma \max Q^*(2, any\ except\ down) = -1 + 0.9 \cdot 0 = -1$$

$$Q_{i+1}(2, down) = Q_i(2, down) + 0.1(Sample - Q_i(2, down)) = -0.1 + 0.1(-1 - (-0.1)) = -0.19$$

Consequently, the Q-value for $Q(2, down)$ is updated once more, yielding a smaller value than in the previous iteration, effectively confirming that moving down from Cell 2 is suboptimal.

	Up	**Right**	**Down**	**Left**
Cell 1	0	-0.1	0	0
Cell 2	0	0	-0.19	0
Cell 3	0	0	0	0
Cell 4	0	0	0	0
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 10: Updated Q-Table after third iteration

The Second Episode

Upon the agent reaching a terminal state, the first episode concludes, and the Q-table undergoes several updates. These updates result in a modified state for the agent at the start of the second episode, which is introduced in the following section.

The 1st Iteration

Keeping the environment the same for each episode; the first iteration starts at state 1. In this instance, the random number $a = 0.829364$ and $\epsilon = 0.936915$, yielding in further exploration. Resulting in a random action, which in this case is "right". Therefore, the agent transitions to cell 2.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 22: State of the agent in the beginning of iteration 1

A Q-value already exists for the state-action pair $Q(1, Right) = -0.1$. The action that

yields the maximum possible Q-value from cell 2 is "Left". Applying the formulas in Equations 27 and 28, the value of $Q(1, right)$ is updated in the Q-table.

	Up	Right	Down	Left
Cell 1	0	-0.1	0	0
Cell 2	-0.1	-0.1	-0.19	0
Cell 3	0	0	-0.19	0
Cell 4	-1	-0.19	-0.1	-0.1
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 11: Q-Table at the beginning of the second episode

	Up	Right	Down	Left
Cell 1	0	-0.19	0	0
Cell 2	-0.1	-0.1	-0.19	0
Cell 3	0	0	-0.19	0
Cell 4	-1	-0.19	-0.1	-0.1
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 12: Updated Q-table after the first iteration

The 2nd Iteration

As a result from the random action before, the agent has now moved to cell 2.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 23: New state of the agent

Following the same process, a randomly generated number a is compared with the current ϵ value. At this point, $a = 352563$ and $\epsilon = 0.932230$, $a < \epsilon$, thus resulting in the random action selection of "up". By observing Table 12 we can deduce that, the current Q-value for the state-action pair $Q(2, up)$ is -0.1. An upwards action from cell 2 would transition the agent to the obstacle cell 6. However, considering the agent cannot enter cell 6, regardless of the action taken, the agent remains in cell 2. In this state, the action that yields a maximum Q-value is "Left", depicted in Table 12. Following the same process by utilising Equations 27 and 28 the Q-table is updated once again using the new value for $Q(2, up)$, as illustrated in table 13, reinforcing that an upwards movement from cell 2 is not a good decision.

The 3rd Iteration

Following the previous action, the agent has remained in cell 2.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 24: State of the agent at the beginning of iteration 3

Generating the random number a again resulted in $a = 0.954214$ and $\epsilon = 0.927569$. Proceeding with the algorithm, the ϵ function has decayed sufficiently, making it more likely to fall below the randomly generated number a, as demonstrated in this case. Resulting in the algorithm to exploit and use the best known action. The best action, determined by the highest Q-value in cell 2, is "Left". In this instance, the state-action pair $Q(2, left)$ is 0. The agent subsequently transitions to cell 1, where the actions that yield the maximum possible Q-value are all actions except "right". By applying the formulas in Equations 27 and 28 the new Q-value for $Q(2, left)$ is determined, visible in table 14.

	Up	**Right**	**Down**	**Left**
Cell 1	0	-0.19	0	0
Cell 2	-0.19	-0.1	-0.1	0
Cell 3	0	0	-0.19	0
Cell 4	-1	-0.19	-0.1	-0.1
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 13: Q-Table after the second iteration

	Up	**Right**	**Down**	**Left**
Cell 1	0	-0.19	0	0
Cell 2	-0.19	-0.1	-0.1	-0.1
Cell 3	0	0	-0.19	0
Cell 4	-1	-0.19	-0.1	-0.1
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	0	0	0
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 14: Updated Q-table after the third iteration

The Third Episode

The agent continues following the same process and iterates until it reaches a terminal state, concluding the second episode. This updates the Q-table several times, modifying the state for the agent at the start of the third episode, as depicted in table 15.

The 1st Iteration

Following the same environment, the first iteration begins at cell 1.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 25: State of the agent at the beginning of iteration 1

In this instance, the random number $a = 0.618287$ is generated and $\epsilon = 0.882220$. $a < \epsilon$, therefore, resulting in the random selection of an action "left". As depicted in Table 15, the Q-value for the state-action pair $Q(1, left)$ is 0. Following the same pattern as before, the Q-value for $Q(1, left)$ is updated as usual, yielding a negative value, as shown in table 16, thus showing that moving to the left is a bad move for the agent when in cell 1.

	Up	Right	Down	Left
Cell 1	0	-0.28	0	0
Cell 2	-0.19	-0.19	-0.361	-0.1
Cell 3	-0.1	-0.28	0	0
Cell 4	-1	-0.19	-0.1	-0.19
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	-1	0	-0.1
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 15: Q-Table at the beginning of the third episode

	Up	Right	Down	Left
Cell 1	0	-0.28	0	-0.1
Cell 2	-0.19	-0.19	-0.361	-0.1
Cell 3	-0.1	-0.28	0	0
Cell 4	-1	-0.19	-0.1	-0.19
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	-1	0	-0.1
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 16: Updated Q-table after the first iteration

The 2nd Iteration

The agent has remained in cell 1 from the previous action.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 26: State of the agent at the beginning of iteration 2

In the second iteration, $a = 0.940884$ while the current value of ϵ is 0.877809. $a > \epsilon$, therefore, the agent undergoes exploitation by transitioning via the best known action. Table 16 suggests the best action is "up" or "down". The agent randomly chooses one of the two. In this case, it chose "up". The Q-value for the state-action pair $Q(1, up)$ is 0. The agent transitions to cell 5, where all Q-values are 0, suggesting an insignificant choice of action for a maximum Q-value. Applying the equations utilised in the previous iterations, the updated Q-value for $Q(1, up)$ is determined, which can be seen in table 17.

The 3rd Iteration

The agent has now transitioned to cell 5 from the previous action.

9	10	11	12 (+10)
5	6	7	8 (-10)
1	2	3	4

Figure 27: State of the agent at the beginning of iteration 3

The randomly generated number a is 0.783432 and ϵ is 0.873420. For this iteration, $a < \epsilon$, therefore, the action is randomly chosen. The action chosen is "right". The updated Q-value for $Q(5, right)$ is computed as usual and displayed in the Q-table (see table 18).

	Up	Right	Down	Left
Cell 1	-0.1	-0.28	0	-0.1
Cell 2	-0.19	-0.19	-0.361	-0.1
Cell 3	-0.1	-0.28	0	0
Cell 4	-1	-0.19	-0.1	-0.19
Cell 5	0	0	0	0
Cell 6	0	0	0	0
Cell 7	0	-1	0	-0.1
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 17: Q-Table after the second iteration

	Up	Right	Down	Left
Cell 1	-0.1	-0.28	0	-0.1
Cell 2	-0.19	-0.19	-0.361	-0.1
Cell 3	-0.1	-0.28	0	0
Cell 4	-1	-0.19	-0.1	-0.19
Cell 5	0	-0.1	0	0
Cell 6	0	0	0	0
Cell 7	0	-1	0	-0.1
Cell 8	0	0	0	0
Cell 9	0	0	0	0
Cell 10	0	0	0	0
Cell 11	0	0	0	0
Cell 12	0	0	0	0

Table 18: Updated Q-table after the third iteration

The 10000th Episode

The steps previously explained were continued until the 10000th episode to ensure convergence is achieved for all Q-values in the Q-table. The best action for each cell is found by selecting the action corresponding to the highest Q-value in its row, depicted in Table 19. The optimal policy is shown in Fig. 28.

	Up	Right	Down	Left
Cell 1	-0.1205	3.122	1.6272	1.4687
Cell 2	2.8431	4.58	2.628	1.4643
Cell 3	6.2	2.5254	4.1753	2.9741
Cell 4	-2.71	-0.2881	-0.2962	4.4326
Cell 5	2.6928	-1.104	-1.1337	-1.0258
Cell 6	0	0	0	0
Cell 7	8	-8.7842	4.4385	6.0719
Cell 8	0	0	0	0
Cell 9	-0.6072	5.5308	-0.631	-0.6146
Cell 10	-0.2727	7.9841	-0.1348	-0.2071
Cell 11	7.7915	10	5.5444	5.6581
Cell 12	0	0	0	0

Table 19: Final Q-table

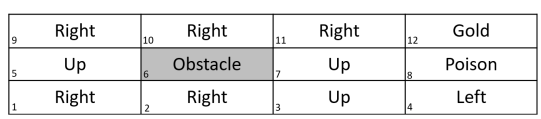


Figure 28: Optimal Policy Grid

The final Q-values show the agent prioritising rightward movement from Cells 1 and 2 to reach the high-reward state in Cell 12. Negative reward cells effectively deter the agent, reinforcing the penalty for entering restricted states. The optimal policy reflects the agent's strategy to reach the gold field in the fewest steps, while avoiding negative states like the poison field and obstacles such as Cell 6.

6 Conclusion

During the process of writing this report and doing the coursework, many valuable insights could be gained in various areas of Machine Learning. To conclude the most important findings, these different areas of Machine Learning have to be considered separately.

The Regression task revealed the importance of architectural design, activation function selection, and types of training algorithms such as SGD and ADAM. While the 1-3-1 network served as a demonstration model, the 1-64-1 network and the deep networks demonstrated a better performance. ADAM's faster convergence and stability emphasised how it is more appropriate in complex regression tasks. Including metrics such as MAE provided a better understanding of the application of such networks practically.

The Classification part showed that using momentum-based optimisers can accelerate the rate of convergence by the equivalent of an order of magnitude's higher learning rate without momentum, although faster convergence does not guarantee optimal or stable training. Furthermore, the experiments demonstrated how architecture size and complexity should match that of the dataset, and the effects on the training dynamics regarding batch size relative to learning rate and stochastic noise from update frequency. It also highlighted the sensitivity of activation function choice, and the possibility of suboptimal convergence due to gradient behavior around 0.

The Transfer Learning tasks showed the impact of network architecture on classification performance with limited training data. GoogLeNet proved superior to AlexNet's simpler architecture, achieving higher accuracy and more stable training dynamic. The hyperparameter analysis demonstrated how modern architectures can maintain consistent performance across different optimisers and parameter combinations, while simpler networks showed higher sensitivity to these choices. This demonstrates how architectural modifications can enhance both accuracy and training robustness in transfer learning applications, particularly when working with constrained datasets.

Q-Learning gave many valuable insights on how an agent can learn optimal actions for navigating a grid world by updating Q-values based on rewards and the Bellman equation. This trial-and-error process demonstrated the importance and ability of the agent to explore and exploit through the ϵ-greedy method to undergo accurate decision-making. Given the results, learning improved the agent's ability to avoid undesired states and reach the rewarding states efficiently. This illustrated how reinforcement learning like Q-learning is robust and adaptable in environments with unknown transition probabilities and reward functions.